WHATNEXT VISION MOTORS

Shaping the Future of Mobility with Innovation and Excellence — Salesforce CRM Project Report

User Story

WhatNext Vision Motors is a pioneering automotive company dedicated to transforming the mobility sector with innovative technology and customer-focused solutions. This Salesforce-based project is designed to enhance the customer experience and streamline operational processes through a centralized system that manages Vehicle, Vehicle Dealer, Vehicle Customer, Vehicle Order, Vehicle Test Drive, and Vehicle Service Request information using Salesforce custom objects and relationships. Record-Triggered Flows automatically assign orders to the nearest dealer and send test drive reminder emails, while Apex triggers and trigger handlers prevent orders for out-of-stock vehicles. Batch Apex and Scheduled Apex jobs update order status to ‘Pending’ or ‘Confirmed’ based on stock availability, giving customers clear and transparent communication about their orders.

1. INTRODUCTION

1.1 Project Overview

The WhatNext Vision Motors Salesforce project is a CRM solution developed to improve the customer ordering process and the company’s operational efficiency. The system automatically suggests the nearest dealer location to customers based on their address, prevents customers from placing orders for vehicles that are out of stock, and runs a scheduled process that updates the status of bulk order records based on stock availability. Customer orders, test drives, and service requests are tracked efficiently within Salesforce, while automation reduces the administrative burden on staff so that employees can focus on strategic tasks that require human expertise.

1.2 Purpose

The purpose of the WhatNext Vision Motors Salesforce project is to:

- Store and manage vehicle details, stock availability, and dealer information in Salesforce.
- Track customer orders, test drives, and service requests efficiently.
- Automatically assign orders to the nearest dealer based on the customer’s address and location.
- Prevent order placement for out-of-stock vehicles, avoiding confusion and disappointment for customers.
- Update order status automatically (‘Pending’ or ‘Confirmed’) based on stock availability.
- Send automated email reminders for scheduled test drives, and scheduled notifications for stock replenishment and order processing.
- Reduce the administrative burden on staff and enable a more agile response to market demands and customer needs.
- Enhance customer satisfaction and loyalty through a more efficient, less error-prone ordering system.

2. IDEATION PHASE

2.1 Problem Statement (Customer Problem Statement Template)

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	Customer	place a vehicle order conveniently	the ordering process takes time and effort	the nearest dealer is not automatically suggested based on my address	Not Provided
PS-2	Customer	order a vehicle	it may be out of stock	orders can be placed without a stock availability check	confused and disappointed
PS-3	Customer	know the status of my order	its status may not reflect stock availability	order status needs to be updated automatically (Pending or Confirmed)	Not Provided
PS-4	Customer	stay informed about my scheduled test drive	reminders may not be sent	reminder communication needs to be automated	Not Provided
PS-5	Staff / Employee	focus on strategic tasks	routine processes demand manual administrative effort	dealer assignment, stock validation, and status updates need to be automated	Not Provided

Problem statements are derived from the use case and expected benefits described in the source. Emotions not stated in the source are marked Not Provided.

2.2 Empathy Map Canvas

User Persona: Customer

- **Think & Feel:** Wants a customer-friendly ordering experience that reduces the time and effort required from the customer.
- **Hear:** Receives automated email reminders for scheduled test drives and clear communication about order status.
- **See:** The suggested nearest dealer location, vehicle stock availability, and order status (Pending / Confirmed).
- **Say & Do:** Places vehicle orders, books test drives, and raises service requests.
- **Pain:** Confusion and disappointment when a vehicle is out of stock; time and effort spent on the ordering process.
- **Gain:** Clear and transparent order status, a more convenient ordering experience, and improved service.

A formal empathy map is not included in the source; the persona above is derived from the use case.

2.3 Brainstorming & Idea Prioritization

Ideas Considered

Not Provided

Final Selection

A Salesforce CRM implementation was selected for the following expected benefits:

- Centralized storage of vehicle, stock, dealer, customer, order, test drive, and service request data
- Automation through Record-Triggered Flows, Apex triggers, Batch Apex, and Scheduled Apex
- Stock validation that prevents orders for unavailable vehicles
- Automatic nearest-dealer assignment and automated test drive reminders
- Reduced administrative burden and improved order-fulfilment accuracy

3. REQUIREMENT ANALYSIS PHASE

3.1 Customer Journey Map

Stages:

- Customer Details Capture (including customer address)
- Vehicle Order Placement (with stock validation)
- Nearest Dealer Assignment
- Order Status Update (Pending / Confirmed)
- Test Drive Booking & Email Reminder
- Service Request

Stages are derived from the process described in the source.

3.2 Solution Requirements

Functional Requirements

FR No	Requirement
FR-1	Storage and management of vehicle details, stock availability, and dealer information
FR-2	Tracking of customer orders, test drives, and service requests
FR-3	Automatic assignment of orders to the nearest dealer based on customer location
FR-4	Prevention of order placement when the vehicle is out of stock
FR-5	Automated email reminders for scheduled test drives
FR-6	Apex triggers and trigger handlers to enforce business rules such as stock validation and automatic dealer assignment
FR-7	Batch Apex job to periodically check vehicle stock levels, update availability, and update bulk order status based on stock availability
FR-8	Scheduled email notifications for stock replenishment and order processing

Non-Functional Requirements

NFR	Description
Usability	Customer-friendly ordering experience that reduces the time and effort required from the customer
Security	Lightning App access granted through the System Administrator profile
Reliability	Accurate order fulfilment and order status that reflects actual stock availability

NFR	Description
Performance	Batch Apex processes order records in batches of 50 on a schedule
Availability	Not Provided
Scalability	Not Provided
Data Integrity	Orders can be created only for vehicles that are currently available
Maintainability	Modular Apex design using trigger handlers that follow best practices
Efficiency	Automation of routine processes to reduce the administrative burden on staff

3.3 Data Flow Diagram

Order Processing Flow

```
Customer
   |
   v
Vehicle Order Created (linked to Vehicle Customer and Vehicle)
   |
   v
Apex Trigger (Before Save): Stock Validation
   |
   v
Record-Triggered Flow "Auto Assign Dealer"
(Get Customer Information > Get Nearest Dealer > Assign Dealer to Order)
   |
   v
Apex Trigger (After Save): Stock Update for Confirmed Orders
   |
   v
Scheduled Batch Apex: Pending Orders Re-checked Against Stock
   |
   v
Order Status Updated to Confirmed
```

Test Drive Reminder Flow

```
Vehicle Test Drive Record (Status = Scheduled)
   |
   v
Record-Triggered Flow "Test Drive Reminder"
(Scheduled Path: 1 Day Before Test_Drive_Date__c)
   |
   v
Get Customer Information
   |
   v
Send Email: "Reminder: Your Test Drive is Tomorrow!"
   |
   v
Customer Receives Reminder
```

3.4 Technology Stack

Layer	Technology
Platform	Salesforce Developer Edition (Developer Org)
UI	Lightning App (WhatNext Vision Motors), Custom Object Tabs, Lightning App Builder
Database	Salesforce Custom Objects (Vehicle, Vehicle Dealer, Vehicle Customer, Vehicle Order, Vehicle Test Drive, Vehicle Service Request)
Business Logic	Apex Classes, Apex Triggers with Trigger Handler
Automation	Record-Triggered Flows (Auto Assign Dealer, Test Drive Reminder) with Scheduled Path
AI	Not Provided
Notifications	Flow Send Email action (automated test drive reminder emails)
Processing	Batch Apex, Scheduled Apex (Schedulable)
Security	Profiles (System Administrator)
Reporting	Reports and Dashboards (added as Lightning App navigation items)

4. PROJECT DESIGN PHASE

4.1 Problem–Solution Fit

Problem	Solution	Benefit
Ordering takes customer time and effort to find a suitable dealer	Record-Triggered Flow that assigns the nearest dealer based on customer address	More convenient, efficient ordering
Orders placed for out-of-stock vehicles cause confusion and disappointment	Apex trigger and trigger handler for stock validation	Orders only for available vehicles
Order status may not reflect stock availability	Batch Apex and Scheduled Apex status updates (Pending / Confirmed)	Accurate, transparent order status
Customers need timely test drive communication	Record-Triggered Flow with scheduled path and email	Automated reminders
Heavy administrative workload for staff	Process automation (Flows, Apex, Batch jobs)	Staff focus on strategic tasks
Vehicle, dealer, customer, and order information must be managed together	Six custom objects with relationships	Structured, centralized data

4.2 Proposed Solution

Parameter	Description
Problem	An inefficient ordering process, orders for out-of-stock vehicles, and manual administrative effort
Solution	A Salesforce CRM system with custom objects, Record-Triggered Flows, Apex triggers, Batch Apex, and Scheduled Apex
Innovation	Automatic nearest-dealer suggestion, stock-based order validation, and scheduled order status updates
Impact	More efficient ordering, fewer errors, better service, higher customer satisfaction and loyalty, and reduced administrative burden
Business Model	Not Provided
Scalability	Not Provided

4.3 Solution Architecture

- **Users:** Customer, Staff, System Administrator

↓

- Salesforce Lightning App — WhatNext Vision Motors
- Data Layer
  - Vehicle
  - Vehicle Dealer
  - Vehicle Customer
  - Vehicle Order
  - Vehicle Test Drive
  - Vehicle Service Request
- Automation Layer
  - Record-Triggered Flows (Auto Assign Dealer, Test Drive Reminder)
  - Apex Trigger and Trigger Handler (VehicleOrderTrigger, VehicleOrderTriggerHandler)
  - Batch Apex (VehicleOrderBatch)
  - Scheduled Apex (VehicleOrderBatchScheduler)
  - Email Reminders
- AI Layer
  - Not Provided
- Security Layer
  - System Administrator Profile

↓

- Reports & Dashboards

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

- Methodology (Agile, epics, story points, sprint-based execution, velocity): Not Provided

- The implementation was organized into seven sequential milestones:

1 Salesforce Developer Account
2 Objects & Relationships
3 Tabs
4 The Lightning App
5 Fields & Relationships
6 Flows
7 Apex and Trigger Batch Jobs

Product Backlog, Sprint Schedule, and Estimation

Sprint	Functional Requirement (Epic)	User Story No.	User Story / Task	Story Points	Priority	Team Members
Not Provided	Milestone 1 – Salesforce Developer Account	Not Provided	Create and activate a Salesforce developer org	Not Provided	Not Provided	Not Provided
Not Provided	Milestone 2 – Objects & Relationships	Not Provided	Create the Vehicle, Vehicle Dealer, and remaining custom objects	Not Provided	Not Provided	Not Provided
Not Provided	Milestone 3 – Tabs	Not Provided	Create custom tabs for all custom objects	Not Provided	Not Provided	Not Provided
Not Provided	Milestone 4 – The Lightning App	Not Provided	Create the WhatNext Vision Motors Lightning App	Not Provided	Not Provided	Not Provided
Not Provided	Milestone 5 – Fields & Relationships	Not Provided	Create fields and relationships for all objects	Not Provided	Not Provided	Not Provided
Not Provided	Milestone 6 – Flows	Not Provided	Create record-triggered flows for dealer assignment and test drive reminders	Not Provided	Not Provided	Not Provided
Not Provided	Milestone 7 – Apex and Trigger Batch Jobs	Not Provided	Create the trigger handler, trigger, batch job, scheduler, and schedule the job	Not Provided	Not Provided	Not Provided

Project Tracker, Velocity & Burndown Chart

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed	Sprint Release Date (Actual)
Not Provided	Not Provided	Not Provided	Not Provided	Not Provided	Not Provided	Not Provided

6. PROJECT DEVELOPMENT PHASE

Phase 1: Requirement Analysis & Planning

Objective

To enhance the customer experience and streamline operational processes by improving the customer ordering process: automatically suggesting the nearest dealer based on the customer’s address,

preventing customers from placing orders for out-of-stock vehicles, and updating bulk order status through a scheduled process (‘Pending’ when the vehicle is out of stock, ‘Confirmed’ when it is in stock).

Key Requirements

1 Salesforce CRM Implementation
   - Store and manage vehicle details, stock availability, and dealer information in Salesforce.
   - Track customer orders, test drives, and service requests efficiently.
   - Automate workflows to assign orders to the nearest dealer based on customer location.
2 Process Automation
   - Prevent order placement if the vehicle is out of stock.
   - Auto-assign orders to the nearest dealer based on the customer’s location.
   - Send automated email reminders for scheduled test drives.
3 Apex and Trigger
   - Implement Apex triggers to enforce business rules such as stock validation and automatic dealer assignment.
   - Use trigger handlers to follow best practices and ensure modularity and maintainability.
4 Batch Jobs
   - Develop a batch Apex job to periodically check vehicle stock levels and update availability.
   - Send scheduled email notifications for stock replenishment and order processing.

Defining Project Scope & Objectives

Project Scope:

1 Build a Salesforce CRM solution for WhatNext Vision Motors to manage vehicles, stock, dealers, customers, orders, test drives, and service requests.
2 Support the ordering process from order placement and stock validation through nearest-dealer assignment and order status update (Pending / Confirmed).
3 Include data model design, automation (Record-Triggered Flows and email reminders), Apex triggers with trigger handlers, Batch Apex, and Scheduled Apex.
4 Provide a Lightning App with navigation for Vehicle, Dealer, Customer, Order, Test Drive, Service Request, Reports, and Dashboard, accessible to the System Administrator profile.

Objectives Summary:

- Create a more efficient ordering system that reduces the potential for errors and improves the overall service provided to customers.
- Enhance customer satisfaction and loyalty by streamlining the ordering process and ensuring accurate stock availability.
- Contribute to operational efficiency by reducing the administrative burden on staff.
- Improve overall productivity and allow a more agile response to market demands and customer needs.
- Provide clear and transparent communication to customers about the status of their orders.

Gathering & Analysing User Needs

Users Involved:

- Customers – Place vehicle orders, book test drives, raise service requests, and receive reminders and order status communication.
- Staff / Employees – Benefit from automated processes and focus on strategic tasks that require human intervention and expertise.
- System Administrator – Configures the Salesforce environment and has access to the WhatNext Vision Motors Lightning App.

Key Functional Needs:

- Capture the customer's address so that the nearest dealer can be suggested automatically.
- Block order placement when the selected vehicle is out of stock.
- Update order status automatically based on stock availability.
- Send automated email reminders for scheduled test drives.
- Send scheduled email notifications for stock replenishment and order processing.

Tools Used:

- Salesforce Developer Org – For configuration, development, and testing.
- Salesforce Flow Builder – To build the record-triggered flows.
- Developer Console – To write Apex classes, triggers, and batch jobs.
- App Manager / Lightning App Builder – To build the Lightning App.

Identifying Key Salesforce Features & Tools Required

- Custom Objects: Vehicle__c, Vehicle_Dealer__c, Vehicle_Customer__c, Vehicle_Order__c, Vehicle_Test_Drive__c, Vehicle_Service_Request__c
- Standard Objects: Not Provided
- Automations: Record-Triggered Flows (Auto Assign Dealer, Test Drive Reminder)
- Apex: Trigger handler class, Apex trigger, Batch Apex, Scheduled Apex
- UI: Custom Object Tabs and Lightning App (WhatNext Vision Motors)
- AgentForce AI & Prompt Builder: Not Provided
- Email Services: Flow Send Email action for test drive reminders
- Security: System Administrator profile
- Skills covered: Data Modelling, Fields and Relationships, Lightning App Builder, Record-Triggered Flows, Apex and Apex Triggers, Batch Apex, Scheduled Apex

Designing Data Model and Security Model

Data Model Includes:

- Vehicle__c (Custom Object)
  - Stores vehicle details.
  - Related to Dealer and Orders.

- Vehicle_Dealer__c (Custom Object)
  - Stores authorized dealer information.
  - Related to Orders.
- Vehicle_Customer__c (Custom Object)
  - Stores customer details.
  - Related to Orders and Test Drives.
- Vehicle_Order__c (Custom Object)
  - Tracks vehicle purchases.
  - Related to Customer and Vehicle.
- Vehicle_Test_Drive__c (Custom Object)
  - Tracks test drive bookings.
  - Related to Customer and Vehicle.
- Vehicle_Service_Request__c (Custom Object)
  - Tracks vehicle servicing requests.
  - Related to Customer and Vehicle.

Security Model Design:

- Role Hierarchy: Not Provided
- Profiles: The System Administrator profile is assigned access to the WhatNext Vision Motors Lightning App.
- Permission Sets: Not Provided
- Record-Level Security: Not Provided
- Field-Level Security: Not Provided in detail; the Dealer lookup field is made Visible to profiles (default setting).

Phase 2: Salesforce Development – Backend & Configurations

This phase focused on developing the core logic of the WhatNext Vision Motors solution. Custom objects were created for Vehicle, Vehicle Dealer, Vehicle Customer, Vehicle Order, Vehicle Test Drive, and Vehicle Service Request with relationships between them. Automation was built using Record-Triggered Flows for dealer assignment and test drive reminders, followed by Apex classes, a trigger, a batch job, and a scheduler (Milestone 5).

Milestone 1 – Salesforce Account

Creating Developer Account

Creating a developer org in Salesforce.

1 Go to https://developer.salesforce.com/signup

2 On the sign-up form, enter the following details:
   - First name & Last name
   - Email
   - Role: Developer
   - Company: College Name

- Country: India
- Postal Code: pin code
- Username: should be a combination of your name and company

The email need not be an actual email id; it can be anything in the format username@organization.com. Click Sign me up after filling in these details.

Account Activation

1 Go to the inbox of the email used while signing up and click Verify Account to activate the account. The email may take 5–10 minutes.
2 Click on Verify Account.
3 Give a password, answer a security question, and click Change Password.
4 You will then be redirected to your Salesforce Setup page.

Milestone 2 – Objects Creation

Navigating to the Setup page: Click the gear icon → click Setup.

Activity 1: Create Vehicle Object

The purpose of creating the Vehicle custom object is to have a clear picture of the vehicle details.

1 From the Setup page → click Object Manager → click Create → click Custom Object.
2 Enter the Label name as Vehicle.
3 Enter the Plural label name as Vehicles.
4 Enter Record Name Label and Format: Record Name → Vehicle Name; Data Type → Text.
5 Select Allow Reports.
6 Select Allow Search → click Save.

Activity 2: Create Dealer Object

The purpose of creating the Dealer object is to have detailed information about the dealer.

1 From the Setup page → click Object Manager → click Create → click Custom Object.
2 Enter the Label name as Vehicle Dealer.
3 Enter the Plural label name as Vehicle Dealers.
4 Enter Record Name Label and Format: Record Name → Dealer Name; Data Type → Text.
5 Select Allow Reports.
6 Select Allow Search → click Save.

Activity 3: Creating Remaining Objects

Create the remaining objects listed in the data model (Vehicle Customer, Vehicle Order, Vehicle Test Drive, and Vehicle Service Request). Their Record Name labels and formats are Not Provided in the source.

Milestone 3 – Fields & Relationships

Key Fields for Each Object

Object	Field	Data Type
Vehicle__c	Vehicle Name	Text
Vehicle__c	Vehicle_Model__c	Picklist (Sedan, SUV, EV, etc.)
Vehicle__c	Stock_Quantity__c	Number
Vehicle__c	Price__c	Currency
Vehicle__c	Dealer__c	Lookup to Vehicle Dealer
Vehicle__c	Status__c	Picklist (Available, Out of Stock, Discontinued)
Vehicle_Dealer__c	Dealer_Name__c	Text
Vehicle_Dealer__c	Dealer_Location__c	Text
Vehicle_Dealer__c	Dealer_Code__c	Auto Number
Vehicle_Dealer__c	Phone__c	Phone
Vehicle_Dealer__c	Email__c	Email
Vehicle_Order__c	Vehicle_Customer__c	Lookup to Vehicle_Customer__c
Vehicle_Order__c	Vehicle__c	Lookup to Vehicle__c
Vehicle_Order__c	Order_Date__c	Date
Vehicle_Order__c	Status__c	Picklist (Pending, Confirmed, Delivered, Canceled)
Vehicle_Customer__c	Customer_Name__c	Text
Vehicle_Customer__c	Email__c	Email
Vehicle_Customer__c	Phone__c	Phone
Vehicle_Customer__c	Address__c	Text
Vehicle_Customer__c	Preferred_Vehicle_Type__c	Picklist (Sedan, SUV, EV, etc.)
Vehicle_Test_Drive__c	Vehicle_Customer__c	Lookup to Vehicle_Customer__c
Vehicle_Test_Drive__c	Vehicle__c	Lookup to Vehicle__c
Vehicle_Test_Drive__c	Test_Drive_Date__c	Date
Vehicle_Test_Drive__c	Status__c	Picklist (Scheduled, Completed, Canceled)
Vehicle_Service_Request__c	Vehicle_Customer__c	Lookup to Vehicle_Customer__c
Vehicle_Service_Request__c	Vehicle__c	Lookup to Vehicle__c
Vehicle_Service_Request__c	Service_Date__c	Date
Vehicle_Service_Request__c	Issue_Description__c	Text
Vehicle_Service_Request__c	Status__c	Picklist (Requested, In Progress, Completed)

Activity 1: Creating a Field in Vehicle Object (Vehicle Model)

1 Go to Setup → click Object Manager → type the object name (Vehicle) in the Quick Find bar → click the object.

2 Click Fields & Relationships → New.

3 Select Data Type as Picklist → click Next.

4 Field Label: Vehicle Model. Enter the picklist values (Sedan, SUV, EV, etc.), each on a new line.

5 Click Next → Next → Save and New.

Activity 2: Creating the Stock Quantity Field in Vehicle Object

1 Repeat steps 1 and 2 from Activity 1.

2 Select Data Type as Number → click Next.

3 Field Label: Stock Quantity (Length: 18, Decimal Places: 0).

4 Click Next → Next → Save and New.

Activity 3: Creating a Lookup Relationship from Vehicle Object to Dealer Object

1 Select Data Type as Lookup Relationship → click Next.

2 Select the Dealer object in the Related To field → click Next.

3 Enter Field Label: Dealer and Field Name: Dealer → click Next.

4 Keep the field-level security as default (Visible) → click Next and Save.

Activity 4: Creating Remaining Objects and Fields

Create the remaining fields and relationships for all objects as listed in the Key Fields table above.

Milestone 4 – Flows (Process Automation)

Flow 1: Record-Triggered Flow to Assign the Nearest Dealer to the Customer's Location

1 In Quick Find, type Flows and click Flows. Click New Flow.

2 Select Triggered Automation, then select Record-Triggered Flow.

3 Select the Vehicle Order object. Trigger the flow when: A record is created.

4 Set Entry Conditions: All Conditions Are Met (AND); Field: Status__c; Operator: Equals; Value: Pending.

5 Click + → select Get Records. Label: Get Customer Information; Object: Vehicle Customer; Condition — Field: Id; Operator: Equals; Value: {!$Record.Vehicle_Customer__c}.

6 Click + → select Get Records. Label: Get Nearest Dealer; Object: Vehicle Dealer; Condition — Field: Dealer_Location__c; Operator: Equals; Value: {!Get_Customer_Information.Address__c}.

7 Click + → select Update Records. Label: Assign Dealer to Order; How to Find Records to Update and Set Their Values: Use the IDs and all field values from a record or record collection; Select Record(s) to Update: {!Get_Nearest_Dealer}.

8 Click Save. Label Name: Auto Assign Dealer.

9 Click Activate.

Flow structure: Start (Record-Triggered Flow, Run Immediately) → Get Customer Information → Get Nearest Dealer → Assign Dealer to Order → End.

Flow 2: Record-Triggered Flow to Send an Email to the Customer Reminding About the Test Drive

1 Select Record-Triggered Flow.

2 Select the Vehicle Test Drive object. Trigger the flow when: A record is created or updated. Set Entry Conditions: All Conditions Are Met (AND); Field: Status__c; Operator: Equals; Value: Scheduled.

3 Click + Add Scheduled Path (below the trigger). Label: Reminder Before Test Drive; Time Source: Test_Drive_Date__c; Offset Number: 1; Offset Options: Days Before. Click Done.

4 Click + Add Element → Get Records. Label: Get Customer Information; Object: Vehicle_Customer__c; Filter Condition: Id = {!$Record.Vehicle_Customer__c}; How Many Records to Store: Only the first record; How to Store Record Data: Automatically store all fields.

5 Send Reminder Email: Click + Add Element → Action. Action Type: Send Email; Label: Send Test Drive Reminder; Subject: "Reminder: Your Test Drive is Tomorrow!"; Recipient Address List: {!Get_Customer_Information.Email__c}; Rich-Text-Formatted Body: True; Use Line Breaks: True.

6 Body: Create a new resource of type Text Template. API Name: EmailSent. Body text is shown below.

Dear {!Get_Customer_Information.Name},

This is a reminder that your test drive for {!$Record.Vehicle__c} is scheduled for tomorrow. If you need to reschedule, contact us at [Support Email]. Thank you, WhatNext Vision Motors Team

7 Click Done and set the email Body to the EmailSent resource.

8 Click Save. Label Name: Test Drive Reminder.

9 Click Activate.

Flow page after creation: the flow canvas shows the Start element with two paths — Run Immediately (Get Customer Information → Send Test Drive Reminder → End) and the scheduled path Reminder Before Test Drive (→ End).

Milestone 5 – Apex and Trigger Batch Jobs

Creating the Trigger Handler Class

1 Click Developer Console from the gear icon.

2 Click File → New → Apex Class.

3 Class Name: VehicleOrderTriggerHandler → click OK.

4 Write the Apex code (source code below).

```
public class VehicleOrderTriggerHandler {

    public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id,
            Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean
            isInsert, Boolean isUpdate) {
        if (isBefore) {
            if (isInsert || isUpdate) {
                preventOrderIfOutOfStock(newOrders);
            }
        }

        if (isAfter) {
            if (isInsert || isUpdate) {
                updateStockOnOrderPlacement(newOrders);
```

```
            }
        }
    }

    // Method to prevent orders when the vehicle is out of stock
    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c
                    WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }

            for (Vehicle_Order__c order : orders) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c <= 0) {
                        order.addError('This vehicle is out of stock. Order cannot be placed.');
                    }
                }
            }
        }
    }

    // Method to update vehicle stock when an order is placed
    private static void updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null && order.Status__c == 'Confirmed') {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c
                    WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }

            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
            for (Vehicle_Order__c order : orders) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c > 0) {
                        vehicle.Stock_Quantity__c -= 1;
                        vehiclesToUpdate.add(vehicle);
                    }
                }
            }

            if (!vehiclesToUpdate.isEmpty()) {
                update vehiclesToUpdate;
            }
        }
    }
}
```

Creating the Apex Trigger

1 Click File → New → Apex Trigger.

2 Name: VehicleOrderTrigger; sObject: Vehicle_Order__c → click Submit.

3 Call the Apex class (trigger handler) from the trigger:

```
trigger VehicleOrderTrigger on Vehicle_Order__c (before insert, before update, after insert,
        after update) {
    VehicleOrderTriggerHandler.handleTrigger(Trigger.new, Trigger.oldMap, Trigger.isBefore,
            Trigger.isAfter, Trigger.isInsert, Trigger.isUpdate);
}
```

Creating the Batch Job

Scenario: A customer places an order, but the vehicle is out of stock. The order remains Pending. After new stock is added, the batch job updates the order to Confirmed.

Create a new Apex class named VehicleOrderBatch:

```
global class VehicleOrderBatch implements Database.Batchable<sObject> {

    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Vehicle__c
            FROM Vehicle_Order__c
            WHERE Status__c = 'Pending'
        ]);
    }

    global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
        Set<Id> vehicleIds = new Set<Id>();

        for (Vehicle_Order__c order : orderList) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c
                    WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }

            List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

            for (Vehicle_Order__c order : orderList) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c > 0) {
                        order.Status__c = 'Confirmed';
                        vehicle.Stock_Quantity__c -= 1;
                        ordersToUpdate.add(order);
                        vehiclesToUpdate.add(vehicle);
                    }
                }
            }

            if (!ordersToUpdate.isEmpty()) {
                update ordersToUpdate;
            }

            if (!vehiclesToUpdate.isEmpty()) {
                update vehiclesToUpdate;
            }
```

```
        }
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Vehicle order batch job completed.');
    }
}
```

Creating the Schedule Class

Create a schedule class named VehicleOrderBatchScheduler that invokes the batch class:

```
global class VehicleOrderBatchScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        VehicleOrderBatch batchJob = new VehicleOrderBatch();
        Database.executeBatch(batchJob, 50); // 50 is the batch size
    }
}
```

Scheduling the Batch Job

Open Debug → Open Execute Anonymous Window and run the following code (the cron expression runs the job daily at 12:00 PM):

```
String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM
System.schedule('Daily Vehicle Order Processing', cronExp, new VehicleOrderBatchScheduler());
```

Verifying the Scheduled Job

In Setup, type Jobs in Quick Find and open Scheduled Jobs to check where the scheduled job is running.

Phase 3: UI/UX Development & Customization

To provide convenient navigation across the WhatNext Vision Motors modules through custom object tabs and a dedicated Lightning App.

Milestone 1 – Tabs

Activity 1: Creating a Custom Tab (Vehicle)

1 Go to the Setup page → type Tabs in the Quick Find bar → click Tabs → click New (under Custom Object Tabs).

2 Select Object (Vehicle) → select any tab style → click Next (Add to Profiles page) and keep it as default → click Next (Add to Custom App) and keep it as default → click Save.

Activity 2: Create tabs for the other objects created above in the same way.

Milestone 2 – The Lightning App

Activity 1: Create a Lightning App

1 Go to the Setup page → search App Manager in Quick Find → select App Manager → click New Lightning App.

2 Fill in App Details & Branding: App Name: WhatNext Vision Motors; Developer Name: auto-populated; Description: a meaningful description; Image: optional; Primary Color Hex Value: keep the default.

3 Click Next → (App Options page) keep the default → Next → (Utility Items) keep the default → Next.

4 Add Navigation Items: search for and move the following items using the arrow button: Vehicle, Dealer, Customer, Order, Test Drive, Service Request, Reports, Dashboard → Next. Select the custom objects created in the previous activities.

5 Add User Profiles: search for the System Administrator profile → click the arrow button → click Save & Finish.

Phase 4: Data Migration, Testing & Security

Data Migration

Not Provided

Testing

The following verifications were performed in the Salesforce Developer Org:

- **Stock validation:** Saving a Vehicle Order (Status: Pending) for an out-of-stock vehicle displays the error “This vehicle is out of stock. Order cannot be placed.” and the order is not saved.
- **Test drive reminder:** After the Test Drive Reminder flow is activated, the customer receives an email addressed to the customer by name, stating that the test drive is scheduled for tomorrow and signed by the WhatNext Vision Motors Team.
- **Scheduled job:** The job Daily Vehicle Order Processing is listed under Setup → Scheduled Jobs with the type Scheduled Apex.

Security

Not Provided

Phase 5: Deployment, Documentation & Maintenance

- **Deployment:** The solution is built and verified in a Salesforce Developer Org. Sandbox, UAT, or production deployment details are Not Provided.
- **Documentation:** The milestone-wise configuration steps and Apex source code are documented in this report.
- **Maintenance, Monitoring & Troubleshooting:** Not Provided. Scheduled jobs can be checked under Setup → Scheduled Jobs.

7. FUNCTIONAL & PERFORMANCE TESTING

Functional testing of the WhatNext Vision Motors solution was performed in the Salesforce Developer Org. The results available in the source are summarized below.

Feature	Test Performed	Result
Stock validation (Apex trigger)	Saved a Vehicle Order for an out-of-stock vehicle	Order blocked with the error: This vehicle is out of stock. Order cannot be placed.
Test drive reminder (Record-Triggered Flow)	Test drive flow activated for a customer	Reminder email received by the customer
Scheduled batch job	Scheduled Daily Vehicle Order Processing using the Execute Anonymous Window	Job listed under Scheduled Jobs as Scheduled Apex
Nearest dealer assignment	Not Provided	Not Provided
Order status update by batch job	Not Provided	Not Provided
Dashboard analytics	Not Provided	Not Provided

Performance testing: Not Provided

8. ADVANTAGES & DISADVANTAGES

Advantages

- Creates a more efficient ordering system that reduces the potential for errors.
- Automatically suggests and assigns the nearest dealer based on the customer's address, reducing the customer's time and effort.
- Prevents orders for out-of-stock vehicles, avoiding customer confusion and disappointment.
- Improves the accuracy of order fulfilment and gives clear, transparent order status (Pending / Confirmed).
- Sends automated email reminders for scheduled test drives.
- Reduces the administrative burden on staff so employees can focus on strategic tasks.
- Improves overall productivity and allows a more agile response to market demands and customer needs.
- Enhances customer satisfaction and loyalty.

Disadvantages

Not Provided

9. CONCLUSION

The WhatNext Vision Motors Salesforce project demonstrates how Salesforce can be used to build a customer-centric vehicle ordering and operations solution. The project integrates custom objects and relationships, Record-Triggered Flows, Apex triggers with trigger handlers, Batch Apex, Scheduled Apex, and email reminders to support nearest-dealer assignment, stock validation, order status updates, and test drive communication.

By streamlining the ordering process and ensuring accurate stock availability, the project is expected to reduce errors, lower the administrative burden on staff, and improve customer satisfaction and loyalty, while giving the company a more agile response to market demands and customer needs.

10. FUTURE SCOPE

Not Provided in the source. The following items are listed in the project requirements, but their implementation steps are not shown in the source:

- A batch Apex job that periodically checks vehicle stock levels and updates vehicle availability.
- Scheduled email notifications for stock replenishment and order processing.

11. APPENDIX

- A. Custom Objects: Vehicle__c, Vehicle_Dealer__c, Vehicle_Customer__c, Vehicle_Order__c, Vehicle_Test_Drive__c, Vehicle_Service_Request__c
- B. Standard Objects: Not Provided
- C. Automation: Record-Triggered Flows (Auto Assign Dealer, Test Drive Reminder), Email Reminders
- D. Apex: VehicleOrderTriggerHandler, VehicleOrderTrigger, VehicleOrderBatch, VehicleOrderBatchScheduler
- E. AI: Not Provided
- F. UI: Custom Object Tabs, Lightning App (WhatNext Vision Motors)
- G. Security: System Administrator Profile
- H. Analytics: Reports & Dashboards (Lightning App navigation items)
- I. Testing: Stock validation, test drive reminder email, scheduled job verification

Thank You